

Draftly Architecture Diagram

```
flowchart TB
    %% === ① Event & Input layer ===
    subgraph L1["① Events & Input"]
        GH["GitHub Webhooks<br/>PR · Issue · Release"]
        SL["Slack Events<br/>Socket Mode"]
        DC["Discord Gateway"]
        CRON["Scheduled Jobs<br/>rq-scheduler"]
    end

    %% === ② Ingestion & Execution layer ===
    subgraph L2["② Ingestion & Execution"]
        API["FastAPI Routes<br/>auth · webhook verification"]
        EVT["Event Composition<br/>→ normalized envelope"]
        RQ["Redis RQ Queues<br/>webhooks · scheduled · de"]
        WRK["RQ Worker"]
        RUN["WorkflowRunner<br/>claim / idempotency · ses"]
    end

    %% === ③ Strands Orchestration layer (the star) ===
    subgraph L3["③ Strands Orchestration – documentation"]
        subgraph G["Strands Graph · GraphBuilder · condit"]
            direction TB
            CL["classify"]
            CX["context"]
            RS["research<br/>(Strands Swarm)"]
            IM["impact"]
            UP["update"]
            CR["create"]
            AN["answer"]
            EVAL["evaluate<br/>(EvaluatorNode)"]
            CH["changelog"]
            CE["changelog_evaluate"]
            DE["deliver"]
        end
    end
```

```

        NT["notify"]
        NP["notify_post"]
    end

    subgraph XC["Cross-cutting Strands plugins on eve"]
        STEER["Steering Handler<br/>Guide · Interrupt"]
        SKILLS["AgentSkills<br/>17 bundled skills"]
    end

    ROUTE["Model Router<br/>RoleAwareModelResolver"]
    SESS["Session Manager<br/>interrupt resume"]
end

%% === ④ Persistence & Integrations layer ===
subgraph L4["④ Persistence & Integrations"]
    DB[("PostgreSQL · pgvector<br/>reviews · evaluation")]
    REDIS[("Redis<br/>queues · streams · cache")]
    EXT["GitHub · Slack · Discord APIs<br/>delivery +"]
end

%% === ⑤ Human layer ===
subgraph L5["⑤ Human layer"]
    PUI["Review Workspace<br/>Next.js"]
    AUTH["Clerk<br/>authentication"]
end

%% Flow: events → ingestion
GH --> API
SL --> API
DC --> API
CRON --> RQ
API --> EVT
EVT --> RQ
RQ --> WRK
WRK --> RUN

%% Flow: execution → graph
RUN --> CL

%% The documentation graph

```

```

CL --> CX --> RS --> IM
IM --> AN
IM --> UP
IM --> CR
AN --> EVAL
UP --> EVAL
CR --> EVAL
EVAL --> |passed| CH
CH --> CE
CE --> |passed + sealed drafts| DE
EVAL --> |"needs revision (of writer)"| AN
EVAL --> |"needs revision (of writer)"| UP
EVAL --> |"needs revision (of writer)"| CR
CE --> |needs revision| CH
IM --> |release · no docs change| CH
IM --> |PR opened · parallel branch| NT
NT --> NP

```

%% Plugins annotate the graph

```

STEER -. "on every tool call / model turn" .-> CL
SKILLS -. "bundled into each agent" .-> CL

```

%% Cross-cutting services feed the graph

```

ROUTE -. "role-based model selection" .-> CL
SESS -.-> RUN

```

%% Delivery + persistence + human

```

DE -. "open PR / post response" .-> EXT
DE == "pause: ReviewGate interrupt" ==> PUI
PUI == "approve / reject · resume" ==> DE
AUTH -. "sessions" .-> PUI
RUN -- "SSE live progress + states" --> PUI
L3 -. "memory grounding · evidence · eval results" .-
RUN -- "persist runs · reviews · evals" --> DB
API -- "state + event streams" --> REDIS

```

%% — classes —

```

classDef strands fill:#ecfeff,stroke:#0891b2,color:#1
classDef draftly fill:#eff6ff,stroke:#2563eb,color:#1
classDef data fill:#f5f3ff,stroke:#7c3aed,color:#4c1d

```

```
classDef external fill:#f8fafc,stroke:#64748b,color:#
classDef human fill:#fffbeb,stroke:#d97706,color:#783
classDef note fill:#f0fdf4,stroke:#16a34a,color:#1453

class API,EVT,WRK,RUN,EVAL,NT,NP draftly;
class CL,CX,RS,IM,UP,CR,AN,CH,CE,STEER,SKILLS,ROUTE,S
class DB,REDIS data;
class GH,SL,DC,CRON,EXT external;
class PUI,AUTH human;
```

Legend

Color	Meaning	Examples
Teal	Strands Agents SDK constructs (Technological Implementation)	Graph, Agent nodes, Steering, Skills, Model Router
Blue	Draftly-owned application code	FastAPI, WorkflowRunner, EvaluatorNode, tool registries
Purple	State / persistence	PostgreSQL · pgvector, Redis
Gray	External systems & webhooks	GitHub, Slack, Discord
Amber	Human-in-the-loop surface	Review Workspace, Clerk auth

How to read the architecture

1. **Events start the work.** GitHub webhooks, Slack events, Discord messages, and scheduled jobs provide the triggers that Draftly handles in the background.
2. **Ingestion makes execution durable.** FastAPI verifies and

normalizes incoming events before Redis-backed RQ queues hand them to a worker and `WorkflowRunner`.

3. **Strands performs the reasoning.** A conditional agent graph classifies the request, gathers context, delegates research to a Swarm, assesses documentation impact, and chooses whether to answer, update, or create content.
4. **Evaluation governs quality.** Drafts must pass groundedness and completeness evaluation. Failed work loops back to the responsible writer; successful work advances toward delivery.
5. **A human controls publication.** `ReviewGate` interrupts the graph before delivery. Approval or rejection in the review workspace resumes the persisted run with the human decision intact.

The solid arrows show execution flow, dotted arrows show cross-cutting services or external actions, and the thick arrows mark the human interrupt-and-resume boundary.